Firmware-Update Observability in a Single-Earbud AirPods 4 ANC Configuration

A macOS UARP Case Study of Asset Discovery, Update Initiation, Device-State Verification, and Bounded Direct-Transport Diagnostics

Thor Thor

Independent Open-Source Researcher, THOR-SEC / Project THOR | ECHO TRACE
ORCID: 0009-0001-6573-385X `codethor@gmail.com`
GitHub: `https://github.com/codethor0` `https://codethor0.github.io/thor-sec/`

Conducted independently on the author's own time. Not sponsored by, affiliated with, or representative of any employer.

Research observation dates: November 6–7, 2026.



AI-generated conceptual cutaway of an open-ear wireless earbud; not a radiograph, device scan, or verified A3056 internal layout.

Abstract

Wireless earbuds operate on embedded firmware, yet their update process is largely automatic and offers limited user-visible diagnostics. This paper reports an independent macOS investigation of an AirPods 4 with Active Noise Cancellation (ANC) configuration in which the right earbud was lost and only the left A3056 remained. The target, Bluetooth product identifier `0x201B`, reported firmware `8B21`; Apple's published release and the macOS A3056 catalog listed `9A348`. The host contained a corresponding Universal Accessory Restore Protocol (UARP) firmware payload and logged updater registration of the remaining earbud and case at `8.1.21.0`. I reconstructed firmware and Bluetooth state, separated a distinct non-ANC AirPods device, evaluated 60 ANC host-state samples under three intervention conditions, and triaged 3,257 updater log records. An initial controlled trial stopped during a locally imposed connected-audio preflight while the earbud was in its charging case. A subsequent manual intervention completed an `airpods-fw` dry run and posted a system-wide Apple update-scan notification; the preserved transcript did not record the scan timestamp, so the duration of the case-closed interval cannot be reconstructed. Two subsequent cached checks reported `8B21`. At 16:49:43 CST on November 7, 2026, `airpods-fw` reported the target connected and on `8B21`; a separate macOS Bluetooth inventory likewise showed the connected `0x201B` target and case on `8B21`. The bounded trial therefore produced no observed firmware-version change, with the dwell time unauditable. A follow-on series of bounded direct-transport experiments (F-05 through F-13), information-only in most runs with three explicitly authorized offer sessions as exceptions, then established device identity over GATT, confirmed the UARP service, and attempted Apple Accessory Protocol (AAP) and UARP information queries from

a native macOS host. From a paired-but-disconnected state at case-lid open, a single L2CAP control channel opened, the AAP handshake was acknowledged, and UARP sync and version-request frames were transmitted, but no valid UARP version reply was received before the accessory closed the channel. In three separate bounded sessions on the same channel, the accessory itself confirmed running firmware `8.1.21.0` and a staged firmware of `0.0.0.0`, acknowledged a single firmware offer referencing the cached `9A348` build in each session, and then produced client-decoded terminal processing responses before any image-data request. The F-08U/V flags `0x0003` admit more than one interpretation under the cited reverse-engineering notes. No image bytes were served and no staging or Apply command was sent. From a connected audio state, the same control channel was repeatedly unavailable. The inspected evidence did not identify an A3056-specific eligibility decision or a confirmed firmware transfer or activation; the significance and cause of the observed terminal offer-processing notifications, including any eligibility decision, remain unresolved. The missing companion earbud remains a testable dependency hypothesis, and third-party analysis of a sibling product indicates that case-mediated update completion requires both buds while host-driven staging does not. The study provides a dated, reproducible account of firmware availability, scan initiation, connected-device verification, direct-transport behavior, and the diagnostic evidence needed to resolve this maintenance edge case.

Keywords: AirPods 4 ANC; A3056; single-earbud maintenance; firmware update; UARP; macOS; MobileAsset; reproducibility; Bluetooth; L2CAP; repairability

1. Introduction

Modern wireless earbuds are embedded computing devices. Their visible features include playback, microphones, environmental noise control, and connection management; their implementation also depends on firmware resident in the accessory. A user may experience the device as ordinary audio hardware while its maintenance process follows software release, discovery, staging, transfer, and activation rules that are largely hidden from the user interface. Apple's public instructions describe automatic updates while AirPods are in a closed, powered charging case within range of a network-connected Apple host [1]. A separate manual firmware-image selection option is not described in those instructions.

This investigation began with a practical failure mode: one AirPods 4 ANC earbud remained after the right earbud was lost. The remaining left unit was operational as a Bluetooth audio accessory, but continued to report firmware `8B21`. The publicly listed release for its model family was `9A348`. Before the study, I had left the earbud charging for extended periods without observing an update. I also noticed changes in noise-control settings and ear discomfort during use. Those observations prompted a forensic examination of the update path rather than a claim about the source of the symptoms.

An important consumer maintenance question follows. If a user loses one earbud but still has a functioning partner and charging case, does the remaining unit continue to receive firmware updates? A negative outcome could arise at several different points: no suitable image, inability to discover the accessory, update-policy eligibility, unavailable companion device, case or lid state, transport failure, or a host-side version record that was not refreshed. A single unchanging version number cannot distinguish these possibilities.

Research objective. This paper reconstructs the observed state transitions for one device and identifies which stages of firmware delivery were visible to macOS. It records the aborted preflight and the subsequent manual scan request, then compares connected-device firmware state before and after the case-closed interval. It then bounds the question further with a series of bounded direct-transport experiments: information-only diagnostics in most runs, plus three explicitly authorized sessions (F-08T, F-08U, F-08V) that each transmitted one firmware offer referencing the cached candidate image. No firmware image was ever transferred, staged, or applied in any run. The work is a single-case empirical study, with data and reproducibility aids selected for public review.

Contributions. The case contributes (1) a model-specific record separating AirPods 4 ANC from a different AirPods 4 device paired with the same Mac; (2) dated evidence of an old installed version alongside a locally cached newer UARP asset; (3) reconstruction of firmware asset-manager and endpoint-registration messages; (4) an ANC host-state intervention series that distinguishes a third-party control request from later native-interface state; (5) quantified triage of updater log noise; (6) a completed, case-closed scan experiment with a connected-device outcome and an explicit success criterion; and (7) a bounded direct-transport diagnostic series establishing that the accessory accepts a single host L2CAP control-channel open from a paired-but-disconnected state, acknowledges an AAP handshake, answers device-side UARP information queries with its own running (`8.1.21.0`) and staged (`0.0.0.0`) firmware records, acknowledges an offer referencing the cached `9A348` build and then emits client-decoded terminal processing notifications before requesting any image data, while the same channel is unavailable from a connected audio state; and (8) read-only structural analysis of the cached `9A348` payload, in which all 17 declared payloads parsed as bounds-valid and the 6.8 MB FTAB payload passed a SHA-256 integrity check. These are method contributions and case findings, not a generalization to every AirPods configuration.

2. Background, Related Work, and Evidence Model

2.1 Firmware and the charging-case update process

Apple publishes firmware versions separately for different AirPods families and provides a standard update process using a paired iPhone, iPad, or Mac, Wi-Fi, Bluetooth proximity, a powered charging case, and a closed lid [1]. The procedure places the earbuds in the powered case, closes the lid, keeps the case near the host, and waits at least 30 minutes; charge state is a prerequisite, and Apple's charging guidance covers the case-power requirement [5]. At the time of this investigation, Apple's AirPods 4 ANC entry listed release `9A348`. Its corresponding release notes described a Custom EQ feature for eligible products; earlier builds also carried general improvements. The existence of a version difference establishes an observable maintenance gap without identifying the contents of every intermediate release.

For users without a network-connected Apple host nearby, the same support document describes setting up an appointment at an Apple Store or an Apple Authorized Service Provider to update the firmware. That is a vendor-documented service update path; it does not, by itself, establish whether the service path accepts a single surviving earbud, and it does not provide a user-selectable firmware image. Apple's repair program documentation is the corresponding service channel reference [6].

For me, the firmware question was practical. The remaining earbud could still connect to my Mac. The unknown is whether its update eligibility is evaluated as an individual accessory, a two-earbud set, a case-linked group, or a combination. Public update instructions use the plural *AirPods* but do not define the single-surviving-earbud edge case. That omission makes empirical verification and vendor clarification useful.

2.2 Related work

Three bodies of prior work bound this study. First, the open-source `airpods-fw` utility [2] can inspect known device firmware, retrieve Apple release and catalog metadata, inspect local asset records, request an accessory scan, and observe macOS updater logs. Its authors explicitly state that the project has not performed or verified an actual firmware installation; it is an observability tool, not a flasher. Second, the Calvados project published a static analysis of AirPods Pro 2 (USB-C) firmware build `9A348` and a reported successful update of that model from `7.1.21` to `9.0.348` performed from a non-Apple (Linux) host over the buds' GATT UARP service, together with an analysis of the case-mediated sibling-update state machine [3]. Third, the

LibrePods project hosts the community discussion of that update, including the before/after device-reported version strings and a reported community bricking incident during a non-Apple update attempt with no Apple recovery path [4].

Two Calvados findings are directly relevant to the missing-companion question, with an explicit hardware boundary. On AirPods Pro 2 (USB-C), the buds' firmware payload carries an Apple-signed manifest binding no device-specific ECID and no nonce, so bud installation does not require a live Apple signing exchange; and the charging case's copy-and-apply logic reads both buds' version records only when both buds are present, while host-driven staging targets the primary bud alone. Every one of those claims is scoped by its authors to AirPods Pro 2 (USB-C), a different product family from the A3056 target of this study. The shared H2 generation and the shared `9A348` train make architectural similarity plausible, but plausible is not established: offer filters, transport tables, and case state machines can differ per model. This study therefore treats the Calvados results as the closest adjacent evidence, cites them wherever they sharpen a hypothesis, and does not transfer them to A3056.

2.3 Evidence stages: from a catalog entry to a confirmed version

I distinguish a firmware update's observable stages rather than treating an available asset as installed software. Figure 1 summarizes the evidence boundary.

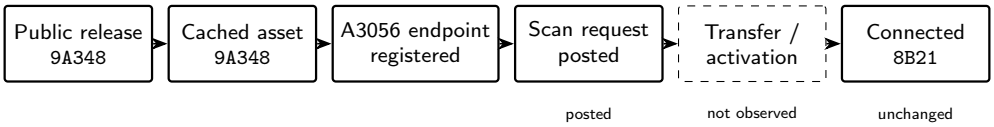


Figure 1: *Observed host-side firmware-update stages for A3056. Solid boxes show recorded events and version readbacks. The dashed transfer/activation stage remained unverified. The connected post-scan firmware readback was `8B21`, not the available candidate `9A348`.*

Definition 1 (Update observation vector)**.** *For a given accessory and observation interval, define*

$$E = (P, C, R, N, T, A, V), \qquad E_i \in \{1, 0, \perp\},$$

where P denotes a published release, C a host-cached candidate payload, R updater registration of the target, N a posted scan notification, T a device-correlated transfer, A an activation acknowledgement, and V a fresh connected-device confirmation of the candidate version `9A348`. The value 1 *means observed,* 0 *means directly excluded by a sufficiently discriminating measurement, and $\perp$ means unobserved or inconclusive.*

This three-state treatment prevents a common forensic error: interpreting a missing log entry as proof that an operation never happened. The final observation vector for the corrected manual trial is $E = (1, 1, 1, 1, \perp, \perp, 0)$. Publication, cache availability, endpoint registration, and a scan request were observed. Transfer and activation remained unverified. The last term records a connected `8B21` readback rather than the candidate `9A348`; it does not establish that the device can never update. The aborted 16:17 preflight is classified separately from the later completed scan request.

Remark. The F-08T/U/V direct-offer sessions have a different observation interval from F-02. Within those bounded client sessions, zero image bytes were served and no Apply command was issued, so transfer and apply were directly excluded for that experimental path; these session-local results do not retroactively change the F-02 scan vector. The accessory reported running `8.1.21.0` and staged `0.0.0.0` at query time. The final offer-processing flags require the separate decoding qualification in Section 6.11.

3. Research Questions and Testable Explanations

The investigation addresses five questions. **RQ1:** Can the host distinguish the surviving A3056 from other paired AirPods and establish its current firmware? **RQ2:** Does macOS have access to the intended newer asset? **RQ3:** What host evidence separates scan requests from actual firmware installation? **RQ4:** Does the available record establish a dependency on the missing right earbud? **RQ5:** Which direct accessory-side transport channels are reachable from an Apple host, and does the accessory answer standards-defined information queries on them?

Table 1: *Candidate explanations, predictions, and required distinguishing evidence.*

Hypothesis	Observable prediction	Discriminating test
H1. Missing companion earbud	Update eligibility blocks or delays a single remaining bud.	Explicit eligibility response or matched complete-pair trial.
H2. Charging or case state	Policy is not satisfied during tested charging periods.	Powered closed-case run with timestamps and post-reconnect check.
H3. Host cache or asset channel	The host has a payload but selects another channel or rejects the subscription.	Asset-selection and rejection trace tied to A3056.
H4. Transport or activation issue	Update starts but fails before persistent activation.	Device-correlated transfer and activation events.
H5. Stale host record	Update occurs but the host still reports a cached version.	Fresh connected readback plus second trusted Apple host.
H6. Transport gating	Accessory-side policy limits which host states may open the update control channel.	Read-only channel probes from discriminating host states.

H1 is the investigation's motivating explanation because the matching right earbud is absent. The evidence permits H1 but has not yet isolated it from H2–H5. The bounded direct-transport series in Section 6.11 was designed to give H4 and H6 discriminating, read-only tests without risking the device. A stronger study needs either an Apple diagnostic decision or a controlled comparison using a complete AirPods 4 ANC pair with otherwise comparable conditions.

4. Experimental Configuration and Data Provenance

4.1 Target identity and comparison-device separation

The target was identified from macOS `system_profiler SPBluetoothDataType` as AirPods 4 with Active Noise Cancellation, Bluetooth product ID `0x201B`, left A3056, firmware `8B21`. The paired right member was missing. The host also displayed a case version string of `8B21`. Analysis of a sibling product line by the Calvados project shows that the charging case runs its own MCU firmware with a distinct version space, and that host-side interfaces do not necessarily report that internal case build identity [3]. The macOS "Case Version" string is therefore treated as a set-level host report of case-associated version state, not as established evidence of the case MCU's own firmware build. A distinct paired AirPods 4 without ANC, Bluetooth product ID `0x2019` and A3053 family, already reported firmware `9A348`. That second device belongs to a third party, is not the missing right member of the target set, and was excluded from the target's result counts.

Table 2: *Device and host configuration at the time of the study.*

Field	Verified status or scope
Target	AirPods 4 with ANC, left A3056; Bluetooth product `0x201B`.
Configuration	One left earbud and its charging case; right earbud unavailable.
Installed release	`8B21` from connected observations before and after the manual scan request.
Host candidate	`9A348` in Apple's public release listing and A3056 catalog.
Charging	User-reported extended charging; a USB-powered closed case was present during later trial.
Host	Apple-silicon Mac; macOS 27.2, build `26B123`; unified log and system-profiler interfaces.
Host-side tools	`system_profiler`, `log show`, `notifyutil`, `airpods-fw`, `SwitchAudioSource`, and `pods-control`.
Control limitations	No matched complete ANC pair, device-side activation trace, or Apple eligibility decision; the two connected readbacks share one Mac host.

4.2 Source corpus and evidence handling

I was the accessory owner and sole operator of the reported experiments. I collected the source material from my own Terminal transcripts, an audio-guard observation CSV, reconnect logs, Bluetooth and accessory diagnostics, cached-asset metadata, and bounded direct-transport experiment logs collected on November 6–7, 2026. Larger historical Bluetooth archives overlap in time and must not be counted as independent experiments. For reproducibility, the exact observable timestamps, product ID, model code, firmware strings, and bounded aggregate counts were preserved. Public exports omit serial numbers, Bluetooth addresses, pairing identifiers, hardware addresses, unrelated account information, and the third-party comparison device's name.

All timestamps shown in the findings use America/Chicago local time, UTC−06:00 on November 7, 2026. A command that returned a cached system-profiler entry was labeled as cached. Such an entry is evidence of what the Mac stored, rather than a fresh device query.

4.3 Intervention controls

No experiment modified the firmware payload, wrote to a UARP cache, disabled system protections, or called an undocumented flashing API. The noise-control tests used a third-party command-line interface for one intervention and a native macOS control for a subsequent intervention. Their readbacks were sampled without corrective action. The charging-case update trial used the macOS accessory scan interface exposed by `airpods-fw` [2]. The ability to post a notification was observed separately from installed-version changes.

The direct-transport series added a stricter baseline rule, *information only*, applying to every run except the three explicitly authorized offer sessions described below. Read-only GATT discovery, device-information reads that excluded the serial-number characteristic, UARP protocol/version information queries, and read-only parsing of the cached payload were permitted; a firmware image transfer, staging, apply/reboot command, or eligibility bypass was never transmitted in any run, and the locally cached Apple-signed firmware image was never modified, rehosted, or redistributed. For each bounded session (F-08T, F-08U, F-08V), I recorded explicit authorization to transmit at most one firmware offer referencing the cached Apple-signed candidate image on an open channel, under a construction in which no transfer, staging, or apply message could follow without a new authorization. In each session the client recorded an offer acknowledgment followed by a terminal processing notification. F-08T was labeled abandoned (reason 0); the raw F-08U/V flags `0x0003`

also admit a denied interpretation under sibling-hardware documentation. No image bytes were requested or served. Several other experiment designs included an explicit operator authorization step and a hard stop before any offer-bearing stage; every one of those runs stopped before that stage or earlier.

5. Methods

5.1 Device discovery and initial firmware baseline

The first step identified devices by Bluetooth product ID and model family, excluding unrelated paired headphones. The command below is an abbreviated reproducible view. Raw output should be retained privately because the underlying system report contains device identifiers.

```
system_profiler SPBluetoothDataType \
  | grep -i -A 30 -B 3 'AirPods' \
  | grep -Evi 'Address:|Serial Number:'
```

At approximately 13:37 local time, the target was connected. The returned version was `8B21` for both earbud and case. Several subsequent `airpods-fw status --device 0x201B` queries recorded the version but explicitly marked the device *disconnected; cached information*. A post-scan reading at 16:49:43 reported the target connected and on `8B21`, and a subsequent independent macOS inventory command also listed the target connected on that firmware. Device identity, cached host state, and connected readbacks were kept separate.

5.2 Audio and listening-mode measurements

My ECHO TRACE audio guard reported selected macOS input and output and queried noise control through `pods-control`. At 15:09:51, the command returned ANC Off. The immediately following 120-second passive series began at 15:10:06 and comprised 24 samples. Following my reported selection through native macOS controls, the next test recorded 24 readbacks across two 12-sample series. A reconnect sequence yielded 12 additional samples. Sample intervals were approximately five seconds for the first observation and two to three seconds for the later checks. The third-party interface was not independently validated against an acoustic or hardware-level measurement.

5.3 Firmware and asset discovery

The `airpods-fw` source code was reviewed locally and its offline tests run before using it. Its `check`, `status`, and `assets --local` subcommands provided the public version, Apple's A3056 catalog metadata, and locally cached payload metadata. A local MobileAsset entry under `/System/Library/AssetsV2/com_apple_MobileAsset_UARP_A3056/` reported a `9A348` payload. Its Info.plist timestamp was August 18, 2026. That date describes a local file modification and cannot establish the date the accessory was updated.

5.4 Update scan requests and host logs

Two separate earlier commands posted the `ScanTriggered` system notification using `notifyutil`. The output confirmed the notification was posted, not that the target was eligible or reached the transfer stage. The utility's `update` function similarly posts one system-wide scan; its `--device` argument filters reporting and observation, not Apple's selection of accessories [2].

I used macOS unified logs for `uarpassetmanagerd`, `uarpd`, and `accessoryupdaterd`. A broad two-hour collection was categorized and then reduced to a 90-second window around target-registration events.

This was a conservative triage of host messages, not a complete device-side trace.

```
log show --start '2026-11-07 15:49:45' \
  --end '2026-11-07 15:51:15' --style compact --info \
  --predicate '(process == "uarpassetmanagerd" OR \
               process == "uarpd" OR \
               process == "accessoryupdaterd")'
```

5.5 First controlled trial and its invalid preflight

At 16:17:47, a controlled trial script was launched. Its preflight used `airpods-fw status --device 0x201B --json` and required the target to be a live-connected macOS audio endpoint before issuing any scan. The remaining earbud was in its USB-powered charging case. The host returned `connected=False` and a cached firmware value of `8B21`; the script stopped without requesting an update. Source review showed this was a local test-protocol condition, not an Apple-reported eligibility rejection. The trial therefore establishes an aborted intervention, not an unsuccessful firmware installation.

5.6 Manual corrected scan and post-scan verification

After the invalid preflight, I ran the `airpods-fw` dry run and then its `update --device 0x201B` command manually. The dry run displayed the following command:

`/usr/bin/notifyutil -p com.apple.SoftwareUpdate.ScanTriggered`

The actual command returned `Scan notification posted`. The left earbud was in its charging case and USB power was connected. The recorded starting batteries were 97% for the earbud and 100% for the case. The exact manual scan timestamp was not printed in the supplied Terminal transcript, which limits elapsed-time precision: bounding the scan to the interval between the 16:17:47 preflight abort and the first post-scan check at 16:47:33 places the subsequent connected verification at 16:49:43 between roughly 2 and 32 minutes after the scan request. Apple's published procedure waits at least 30 minutes with the case closed and powered [1]. The preserved record therefore cannot demonstrate that the trial satisfied that dwell requirement, and the outcome is classified as *no observed version progression with unauditable dwell time*, not as a protocol-complete negative result.

At 16:47:33 and 16:48:27, `airpods-fw` still returned `8B21` with an explicit disconnected/cached label. At 16:49:43, the same utility identified the target as *connected*, reporting `8B21` for both earbud and case, each at 100% battery; the rise from the recorded 97% starting level is consistent with charging in the powered case during the interval. A separate `system_profiler SPBluetoothDataType` inventory, run immediately afterward without a displayed timestamp, reported `Target 0x201B: CONNECTED`, `Firmware Version: 8B21`, and `Case Version: 8B21`. Both host interfaces agreed during the connected verification session. Because they operate on the same Mac, this is cross-interface confirmation, not two independent device-side firmware attestations. The reconnect series also recorded an input-route change to the target AirPods, which is the strongest internal evidence that the reconnect itself succeeded.

The manual intervention was performed directly with `airpods-fw`; the public shell script is an equivalent reproducibility aid and was not the program that posted this successful scan. No additional scan request or direct firmware write was performed during the final verification.

5.7 Bounded direct-transport diagnostics

The evening series (17:21–19:49 local) tested RQ5 with gated direct-transport probes in three families, information-only in most runs and offer-bearing only in the three explicitly authorized sessions (F-08T, F-08U, F-08V).

Read-only GATT identity and service discovery (F-05 through F-05E). A macOS CoreBluetooth scanner enumerated advertising candidates; a GATT client then performed service and characteristic discovery, and Device Information reads restricted to the vendor, product, and model characteristics, explicitly excluding the serial-number characteristic. A combined UARP diagnostic finally performed bounded UARP sync and protocol-version information writes, gated on candidate uniqueness, Apple manufacturer data, and PnP/model identity match.

Native Apple-host transport attempts (F-06, F-08 through F-11). Locally compiled macOS IOBluetooth probes targeted the Apple Accessory Protocol control channel at L2CAP PSM `0x1001`, with the accessory's exact Bluetooth address verified against the paired `0x201B` record before every run. The locally cached Apple-signed A3056 `9A348` image (8,540,839 declared bytes, 17 payloads, an embedded Apple signature present but not independently cryptographically verified, never modified) was present on the host as the candidate an offer would reference, so that a channel which did open could be exercised immediately under its real preconditions. Gate variants added a battery-telemetry precondition (F-08X) and a conditioned offer-window protocol (F-09: iPhone Bluetooth disabled, bud seated in a USB-powered case, lid closed for ten seconds then opened, probe launched within seconds, lid held open). Harness iteration was preserved in the record: an early F-08 variant stopped on an unexpected UARP payload-table structure (F-08), a name-verification variant found zero paired devices matching the verified A3056 name (F-08R), and an address-verification variant stopped on an unexpectedly resolved device address (F-08S). The Calvados reference probe could not be exercised on this host (Darwin arm64 without `dbus` or `bluetoothctl`; the Docker Desktop virtual machine exposes zero Bluetooth adapters, HCI socket unavailable), which bounds reproduction of the sibling project's GATT transfer path on this machine (F-06). Three sessions (F-08T, F-08U, F-08V) carried an explicit recorded operator authorization each to transmit at most one UARP firmware offer on an open channel; F-08U and F-08V added an AACP settle gate holding the open channel for at least six seconds before any UARP traffic. Every authorized session was constructed so that no transfer-request, staging, or apply message could follow the offer without a new authorization, and the UARP Apply message (`0x0A`) was never transmitted in any run. Later conditioned windows (later F-08T repetitions and all F-09 windows) never opened the channel and therefore sent nothing.

Paired-disconnected single-channel probes (F-12, F-12B, F-13). A final family removed all host-initiated connect/disconnect churn: the probe armed while the paired target was disconnected, and opened exactly one L2CAP channel at lid open, transmitting at most an AAP control handshake, an AACP notification registration, and UARP sync and version-request frames. A diagnostic variant (F-13) repeatedly observed the paired connection state with the case lid closed.

All timestamps, target states, actions, and outcomes of the preserved runs are consolidated in Table 4 and Appendix B. Information-only runs transmitted no firmware offer; the three authorized offer sessions each transmitted one offer referencing the cached candidate image, followed by an acknowledgment and a terminal processing notification. The F-08U/V flags `0x0003` are not uniquely classifiable from the released evidence. No run transmitted a firmware transfer, staging, or apply command, and the installed image was never modified.

6. Results

6.1 The firmware-version gap was reproducible

The connected target reported 8B21, while Apple's published AirPods 4 ANC release and its A3056 asset catalog listed 9A348 [1]. The unrelated non-ANC AirPods 4 also reported 9A348, but served only as a model-separation check. Its version was never substituted for that of the target. A connected post-scan reading at 16:49:43, corroborated by a subsequent connected macOS Bluetooth inventory, still reported 8B21. End-of-day inventories on November 7 continued to show the connected 0x201B target on 8B21 (earbud and case), and the separate 0x2019 device on 9A348; the model separation held across the entire study.

6.2 The A3056 update asset was already present on the host

Two separate local sources reported the candidate. One was a MobileAsset metadata object identifying 9A348 as a payload present on the host. The second was a UARP asset-manager record at 15:49:59 and again at 15:50:51, pointing to

```
/var/db/accessoryupdater/assetcache/mobileasset/legacy/A3056/9.0.348.0/prod/firmware.uarp
```

The asset catalog reported an approximately 4.92 MiB package-size field, distinct from the 8,545,519-byte local firmware.uarp file measured below; the original catalog size and extracted file size should not be treated as equivalent. Payload presence and cache-record existence establish that update discovery progressed as far as the host asset layer. The selected transfer channel and whether the earbud ever received image bytes remain unknown.

6.3 Experiment F-04: the cached payload is structurally complete

Read-only parsing of the cached payload reported UARP asset format 3, version 9.0.348.0, with a declared length of 8,540,839 bytes against a physical file length of 8,545,519; the trailing 4,680 bytes form an NSKeyedArchiver property list of 99 objects. All 17 declared payloads parsed as bounds-valid: ten Apple updater payload tags (AUCH, AUDT, AUER, AUFM, AUFR, AUPC, AUPS, AUUF, AUUT, AULB), CMAP, FTAB, MAGN, NERF, STFW, and two TMAP payloads. The FTAB payload (6,795,264 bytes) matched its embedded SHA-256 integrity hash; the STFW payload carried version 90.390.0.0; the NERF payload occupied 342,560 bytes. The accompanying MobileAsset metadata identified the bundle as com.apple.accessory.updater.payload.A3056, build 9A348, and the payload file's SHA-256 (be5fbc6f...fb1d1f) was independently re-confirmed in a later stage of the study. Nothing in this inspection modified the image. The checks establish structural parseability and internal digest consistency for the specific conditions tested; they do not independently establish complete cryptographic authenticity, installability, or the absence of every possible corruption condition. Within those limits, the unresolved stage boundary narrows to selection, eligibility, transfer, or activation rather than the asset-integrity conditions examined here.

6.4 UARP recognized target-associated endpoint records

At 15:50:51, uarpassetmanagerd logged an A3056 endpoint record for the left earbud with fwVersion=8.1.21.0, as well as a case-associated A3056 record with the same normalized version. The same interval reported that a 9.0.348.0 cache record was available for the accessory. These are unusually informative observations because the host processed the old installed version and newer candidate

asset in the same bounded period. They still do not reveal a selected transfer, device acknowledgement, activation, or eligibility decision.

6.5 Connected-link telemetry corroborated the host-reported state

Live accessory-daemon records for the connected target, filtered to the target product identifier, were sampled under three link/activity states. Sample A comprised eight records, each reporting firmware `8B21` with SCO audio active, ANC engaged, advertising-channel use split across channels 37 and 39 (five and three records), and RSSI between −53 and −52 dBm (mean −52.4 dBm). A music-playback sample comprised 17 records, all on channel 37, all reporting `8B21` with ANC, with SCO present in seven records and absent in ten; RSSI ranged from −48 to −39 dBm (mean −43.76 dBm). A third sample of 18 records split six SCO and twelve idle, all in Normal listening mode. The same live record reported Bluetooth version 6.0, the feature-flag set (AACP, MagicPaired, LidClosed, FindMyManaged, SmartRouting, SensorData, ANC, Transparency), a 24 kHz HFP audio route with AAC-ELD codec and a 150 ms jitter buffer on a BCM_4388 host controller, and battery telemetry of 97% for the left bud with 0.0% for the absent right bud, a cached field that also produced phantom 1% and 57% readings later in the day (Section 6.11), all treated as unreliable. Notably, the accessory-side record carried an internal case version string `B435`, distinct from the set-level `8B21` the host displays, corroborating the case-version caveat of Section 4. These samples are host-side link observations; they are not radiofrequency exposure measurements.

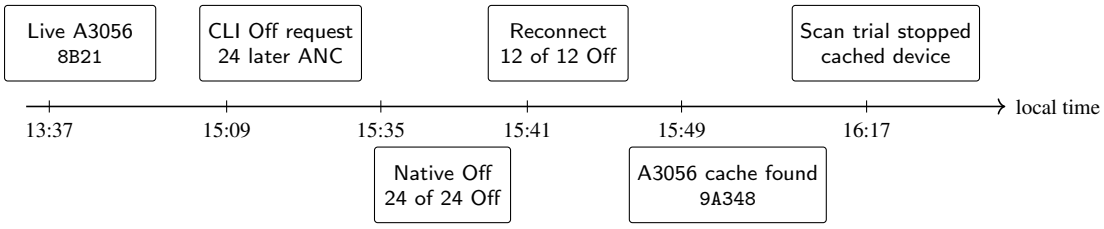


Figure 2: *Selected observations through the first controlled preflight on November 7, 2026. Horizontal spacing is schematic and not to scale. Boxes mark host observations, not continuous monitoring.*

6.6 ANC host readbacks differed by intervention

The first command-line attempt reported Off immediately after applying the setting. Fifteen seconds later, the first passive observation returned Noise Cancellation, and all 24 samples in the two-minute window continued to report that state. In contrast, after a native-interface intervention, 24 of 24 subsequent samples reported Off. The GUI action itself was not time-stamped in the terminal transcript. After a successful reconnect, 12 of 12 samples still reported Off. Two earlier reconnect attempts were incomplete because the script saw only MacBook Pro Speakers. Figure 3 reports the valid sample counts by condition, without implying sample independence.

The comparison shows that ANC Off was stable for the observed later windows and across one successful reconnect. Thus, the early discrepancy cannot be interpreted as evidence that ANC invariably re-enables after reconnection. It is consistent with a third-party setting/reporting issue, conditional behavior, or unobserved state changes. Firmware causation was not established.

6.7 Log volume was dominated by unrelated or ambiguous messages

One two-hour updater extraction included 3,257 records. A text-based triage excluded 1,515 messages of the form `Could not remove expired file at <private>; error <private>` from the `uarpd` host

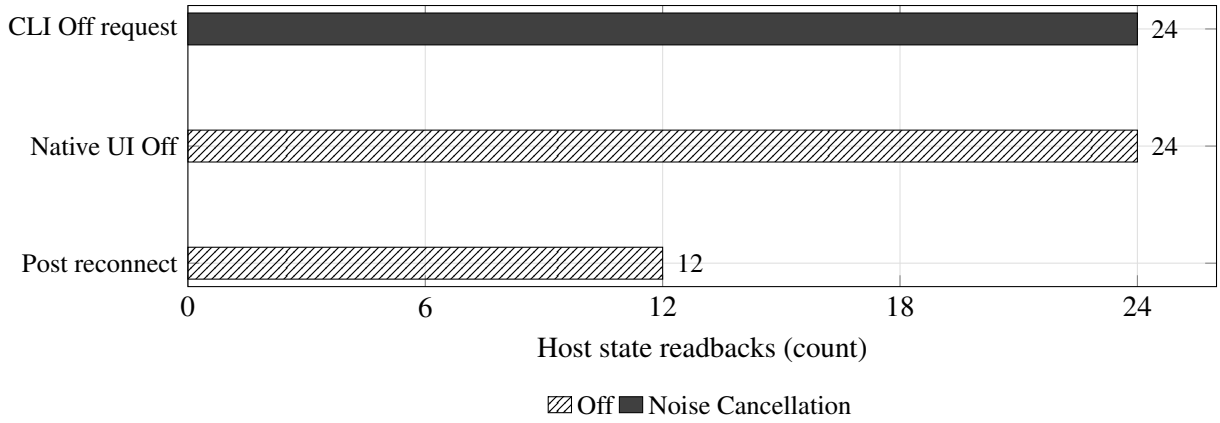


Figure 3: *Sixty host-reported ANC samples under three different observation conditions. Counts indicate repeated reports, not independent trials or independently measured acoustic states.*

pruner. Another 1,531 messages matched a broad keyword expression, with 211 remaining unmatched. The pruner reports were retained in private evidence, but their filenames and error details were redacted by the operating system. They could not be correlated to the target earbud.

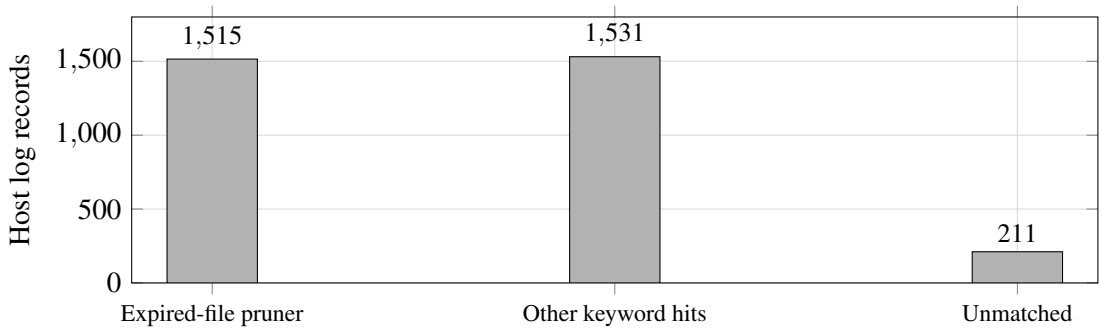


Figure 4: *Host updater log triage. All 3,257 records were partitioned by a documented text filter. These are diagnostic categories, not a measure of defect frequency or device-specific fault attribution.*

Some records reported that `com.apple.AUDeveloperSettings.plist` was unavailable. Other subscription lookups reported `Found asset=(null)` or `No cached record found`, followed by successful lookups of the same A3056 `9A348` cache record. The messages may reflect different subscriptions or fallback paths; the available excerpt is insufficient to attribute a specific updater failure. A case-associated log entry contained a generic `AirPods Pro Case` label, which was treated as a host display label rather than proof of case-hardware mismatch.

6.8 Historical capture triage (November 6 corpus)

Two overlapping same-day historical captures (a primary archive and a follow-up extraction) together contained approximately 230 MB uncompressed and 1,467,868 lines. Because the captures overlap in time, they are treated as one corpus, not as independent replications. Within it, explicit `8B21` mentions numbered 550 in the primary capture and 128 in the follow-up (Figure 5); `periodicFirmwareCheck` matches numbered 4,291 and 88 under the keyword counting method, while a day-level recount for November 2–6 produced 758, 763, 1,290, 836, and 644 matches (Figure 6); the day-level bars sum to the same 4,291 total and show how the periodic checks distribute across days; 133 primary-capture records reported Bluetooth SQL compilation

failures and 10 reported legacy accessory-updater initialization failures; and `9A348` appeared zero times. The zero-count finding is staged evidence: across the historical window the host updater log shows repeated periodic checks against the then-installed version, with no record of the newer candidate being evaluated for this target. A BLE observation window in the same corpus logged advertising activity across channels 37, 38, and 39 (315, 394, and 409 observations; Figure 7); because the observer did not resolve device identity to the AirPods target, those counts describe the surrounding radio environment, not the target's transmit power.

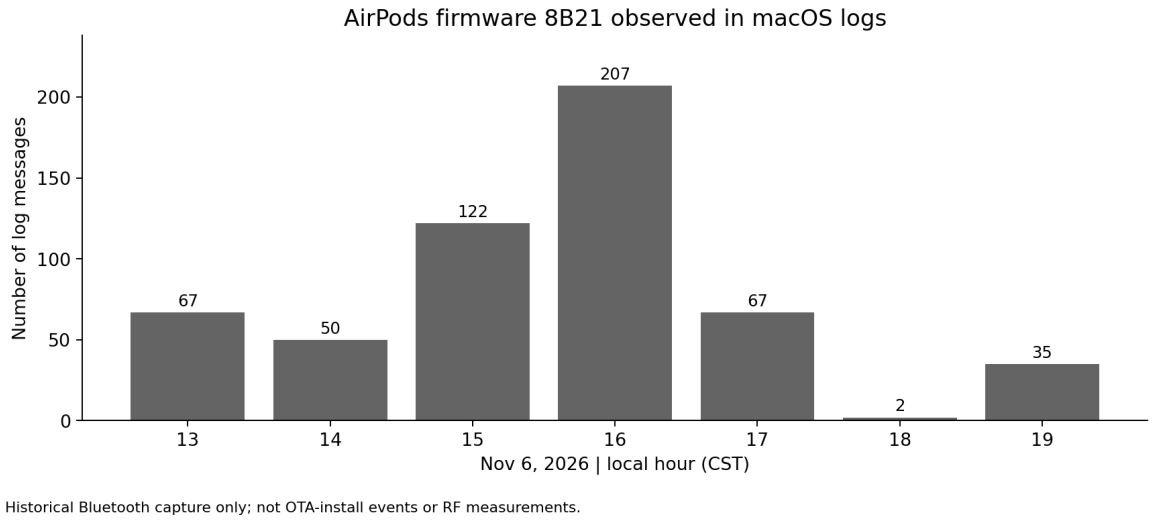


Figure 5: *Hourly count of explicit `8B21` mentions in the November 6 primary historical capture (67+50+122+207+67+2+35 = 550). Counts are log-line mentions, not update events.*

6.9 The first controlled scan trial was blocked by its own preflight

At 16:17:47, the research trial produced:

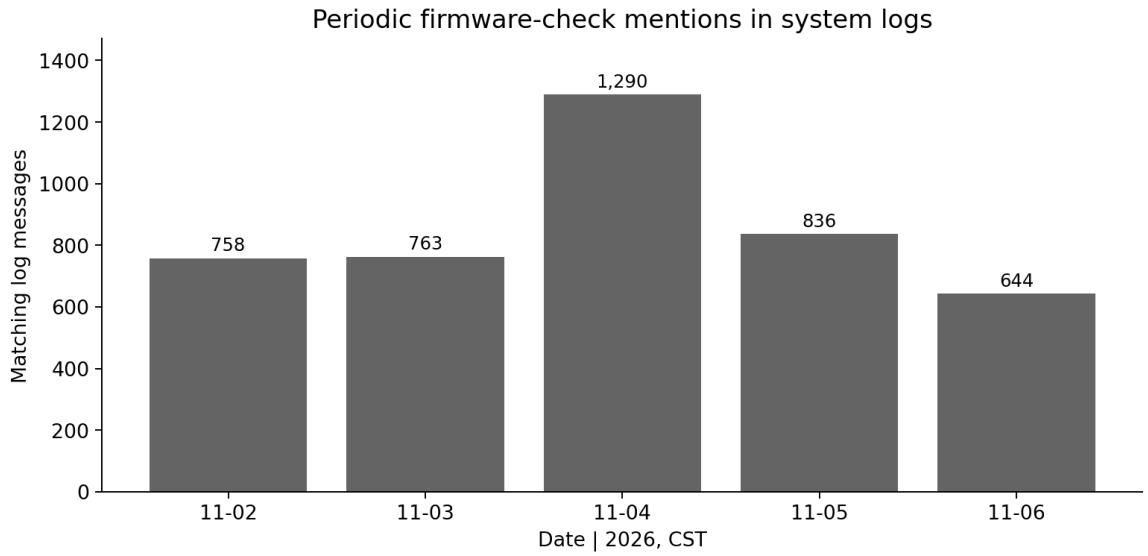
```
Product: 0x201B | Live connected: False |
Installed firmware: 8B21
STOP: macOS reports this device DISCONNECTED (cached reading).
Preflight did not pass. No scan request was posted.
```

The scan script required an active macOS audio-device connection before posting a scan. When the bud was in the closed case, that prerequisite was not met. Apple's published process requires initial pairing/connection but then explicitly moves the earbuds into a closed, powered case for the maintenance interval [1]. The failed preflight therefore exposed a protocol-design problem in the research harness. It did not produce an Apple eligibility verdict and did not test an over-the-air transfer.

6.10 Experiment F-02: scan posted, connected firmware unchanged

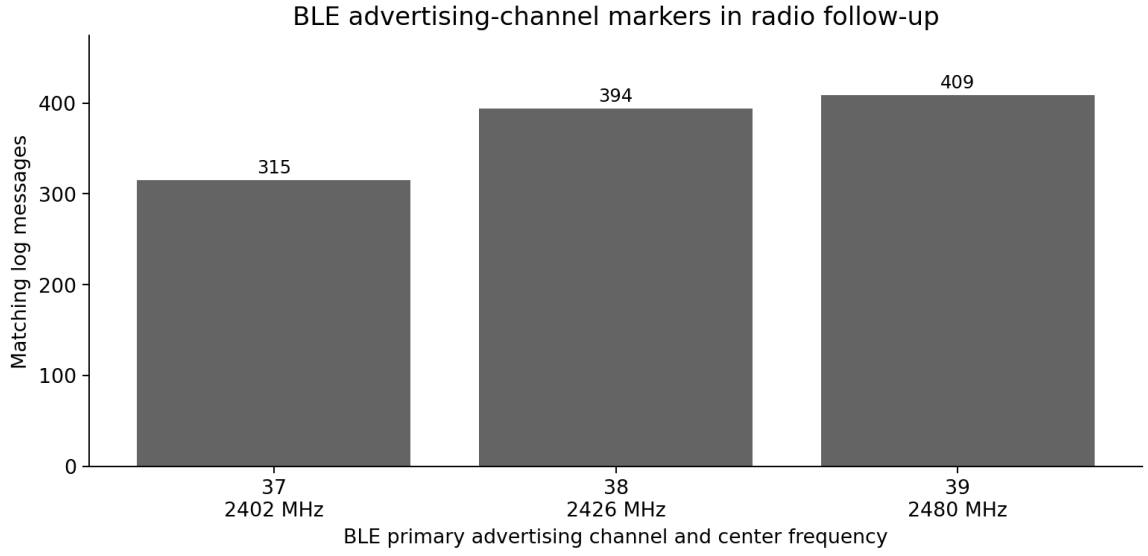
The corrected manual trial progressed beyond the earlier preflight. Table 3 preserves the distinction between an update request, cached queries, and a subsequent connected-device reading.

The observed intervention outcome was *no firmware-version progression during the bounded trial*. The version was obtained from a connected session following a successfully posted scan request. The available host log excerpts did not establish an A3056-correlated transfer attempt, completed activation, or explicit



Matches for periodicFirmwareCheck; not the number of unique checks or AirPods updates.

Figure 6: *Daily `periodicFirmwareCheck` matches, November 2–6 (758, 763, 1,290, 836, 644). The day-level bars sum to 4,291, matching the keyword-level total in the same corpus; the two distributions differ by day, not by total.*



Nearby BLE observations: device identity not resolved to the AirPod; NOT RF power or occupancy.

Figure 7: *Advertising-channel distribution across a BLE observation window (channel 37: 315, channel 38: 394, channel 39: 409). Device identity was not resolved to the AirPods target; these counts describe the RF environment, not the target's transmit power.*

Table 3: *Experiment F-02: bounded post-scan observations on November 7, 2026 (CST). The scan timestamp was not printed in the supplied output.*

Time	Host interface	Observation
16:17:47	Original preflight script	Disconnected/cached `8B21`; script stopped; no scan posted in this first trial.
Not recorded	Manual `airpods-fw update`	One system-wide Apple scan notification successfully posted.
16:47:33	`airpods-fw status/check`	Disconnected/cached; earbud and case `8B21`; 100% battery.
16:48:27	`airpods-fw status`	Disconnected/cached; earbud and case `8B21`.
16:49:43	`airpods-fw status`	Connected target `0x201B`; earbud and case `8B21`; 100% battery.
Immediately after	`system_profiler`	Connected target `0x201B`; earbud and case `8B21`.

rejection decision. The missing companion earbud is one explanatory hypothesis requiring an eligibility trace or a controlled comparator.

6.11 Bounded direct-transport diagnostics: what the accessory answered

Table 4 summarizes every preserved direct-transport run. Four findings survive scrutiny.

Table 4: *Preserved direct-transport runs, November 7, 2026 (CST). Every run was information-only except the three operator-authorized offer sessions (F-08T/U/V), which each transmitted one offer referencing the cached candidate image; acknowledgments and terminal processing events were client-decoded, and F-08U/V flags require qualified interpretation; no run transmitted a transfer, staging, or apply command.*

Time	Experiment	Target state	Outcome
17:21–17:39	F-05, F-05B, F-05C	Advertising / GATT connected	One Apple-manufacturer candidate labeled as AirPods (RSSI −43 dBm in the public CSV); UARP characteristic found under service `4715650B-5E9D-4AC2-B898-A4FC0AA5DF78`; PnP ID `0x004C/0x201B` exact match; model A3056; firmware characteristic present but not decoded.
17:35	F-05D run 1	GATT connected	Six UARP information frames transmitted (sync, protocol version, running tag 5, staged tag 6, update statistics tag 7, last error tag 8); zero replies; session closed by gate.
17:38	F-05D run 2	Two candidates	Uniqueness gate failed; stopped before any write.
17:39	F-05E	GATT connected	UARP sync and protocol-version writes acknowledged (2 ACK, 0 rejected); no UARP indication received; session closed by gate.

(continued on next page)

Table 4 (continued)

Time	Experiment	Target state	Outcome
(not preserved)	F-04, F-06	Host-side	F-04: read-only cached-payload parse, 17/17 payloads bounds-valid, FTAB SHA-256 match. F-06: Calvados reference probe not runnable on this host (no `dbus/bluetoothctl`; Docker VM exposes zero Bluetooth adapters).
18:15	F-08T	Paired, disconnected at lid open	L2CAP PSM `0x1001` open (MTU 672) at 5.0 s; AAP handshake TX; UARP sync RX; protocol v4 negotiated; accessory answered queries: running firmware `8.1.21.0`, staged `0.0.0.0`; `9A348` offer TX; **offer acknowledged; client-decoded abandoned** (reason 0); channel closed at 7.7 s; no image data requested.
18:29	F-08U	Paired, disconnected; AACP settle gate ($\geq$6 s)	Same session structure; running `8.1.21.0`, staged `0.0.0.0` confirmed by accessory; offer TX; **acknowledged; terminal flags** `0x0003` (denied/abandoned ambiguous); no image data requested.
18:30	F-08V	Same as F-08U	Same raw terminal flags; third acknowledged offer without image-data request.
18:49–19:04	F-08T later windows (3), F-09 windows (5)	Connected / conditioned lid window	Every L2CAP open failed `0xE00002BC` (18:56:45, 19:02:30, 19:04:26 for the later F-08T windows); two F-09 windows stopped at the connected-target check; nothing sent.
18:59	F-08X gate	Disconnected	Battery-telemetry gate found zero connected `0x201B` records; stopped before transport.
18:59–19:00	F-10 check/offer	Paired, disconnected	Native baseband reconnect attempt returned `0xE00002D6` after 20.1 s; no UARP offer.
19:07	F-11	Connected	Single L2CAP open attempt failed `0xE00002BC` at 3.2 s; no retry, no disconnect issued.
19:10	F-12	Connected	Precondition check: target already connected; no traffic.
19:12	F-12B run 1	Paired, disconnected	One L2CAP PSM `0x1001` open succeeded (outgoing MTU 672); AAP control handshake send failed `0xE00002E7`; channel closed.
19:13	F-12B run 2	Paired, disconnected	Channel open (MTU 672); AAP handshake transmitted and acknowledged; AACP notification registration, UARP sync, and UARP version request transmitted; no valid UARP version reply; accessory closed the channel at 21.5 s.
19:17–19:49	F-13 (6 attempts)	Lid closed	Target remained CONNECTED through each 120 s wait; never entered DISCONNECTED; no L2CAP or UARP traffic.

First, **device identity is unambiguous over GATT.** The PnP ID read returned vendor `0x004C` and product `0x201B`, the model string contained A3056, and the serial-number characteristic was never queried. The UARP GATT characteristic was present with write and indication properties. A two-candidate scan failed the uniqueness gate and was stopped, which is the intended behavior when a second Apple audio accessory is advertising.

Second, **the accessory accepted writes but returned no UARP indication within the bounded observation window.** In F-05E, both information writes were acknowledged at the ATT layer and none were rejected, yet zero UARP indications arrived during the bounded response window; the experiment does not establish that a response is impossible under other conditions. An acknowledged GATT write establishes transport delivery to the accessory's BLE stack, not that the accessory's UARP state machine consumed the message.

Third, **control-channel reachability depends on host connection state.** From a connected Apple-host audio state, the L2CAP AAP control channel was unavailable across every variant (single open, retry loops, conditioned lid-window protocols), with a consistent IOKit failure result. From a paired-but-disconnected state, channel opens did succeed: in the minimal F-12B design, the case lid opening triggered the open, the AAP handshake was acknowledged on the second attempt, and UARP sync and version-request frames were transmitted, but no valid UARP version reply arrived before the accessory closed the channel 21.5 seconds after the open; the fuller F-08T/U/V sessions on the same channel were answered end to end (detailed in the next subsection). A final diagnostic showed the target holding a CONNECTED state for more than two minutes with the case lid closed, so the paired-disconnected precondition itself was the scarce resource.

Throughout the series, cached right-earbud battery telemetry drifted between unavailable, a phantom 57% at approximately 18:40 (reported for a bud that is physically lost), and 1% across runs. That instability is consistent with a missing companion and confirms that host battery telemetry is not proof of physical earbud presence; it was treated accordingly, motivated the right-bud-gated F-08X design, and was never used as an eligibility signal.

6.12 Experiments F-08T/U/V: answered UARP sessions and terminal offer-processing events

Fourth, **the accessory answered the session and issued terminal offer-processing notifications after acknowledging the offers.** Three authorized sessions reached a stage no earlier run reached: a live, answered UARP session with the accessory's own endpoint. In F-08T (18:15:09), the control channel opened 5.0 seconds after lid-open from a paired-but-disconnected state (outgoing MTU 672, after two short disconnects). The AAP handshake was transmitted, a UARP sync was received from the accessory, protocol version 4 was reported as negotiated (the host-proposed version is not recoverable from the public supplement), and the accessory *answered* the information queries: it reported running firmware `8.1.21.0` and staged firmware `0.0.0.0`, the first device-confirmed version readbacks of the study; the numeric UARP version `8.1.21.0` is consistent with the short `8B21` build designation under a naming convention documented for sibling hardware [3]; this format correspondence is not firmware-installation attestation. The staged-version field `0.0.0.0` records what the accessory reported at query time and does not by itself exclude an earlier or out-of-band staging operation. The offer I authorized referencing the cached `9A348` build was then transmitted; the accessory **acknowledged the offer** and the client subsequently decoded an *offer abandoned* message (reason 0), closing the channel 7.7 seconds after open. No transfer request followed: the accessory never asked for image data. The protocol events of these three sessions were reconstructed from the experimental client's own logs and decoder; an independent full HCI or PacketLogger capture of these exact sessions was not obtained, so the recorded messages are client-decoded observations rather than independently corroborated on-air captures.

F-08U (18:29:03) and F-08V (18:30:33) repeated the session with an added AACP settle gate that held the open channel for at least six seconds before any UARP traffic, to test whether the earlier client-classified terminal response was sensitive to session timing. Both sessions again produced running `8.1.21.0`, staged `0.0.0.0`, an acknowledged offer, and a terminal processing notification with flags `0x0003`, which the local client labeled abandoned. The value is not uniquely classifiable from the released evidence: the Calvados UARP notes also describe a denied status combined with reason 1 as `0x0003` [3]. The settle gate did not change the recorded raw value. In all three sessions the client recorded a terminal offer-processing event before the transfer stage; no image bytes moved in either direction.

The preserved record also shows the harness converging on this result: an early F-08 variant stopped on an unexpected UARP payload-table structure, F-08R stopped when zero paired devices matched the verified A3056 name, F-08S stopped on an unexpectedly resolved device address, and every later conditioned attempt (later F-08T windows at 18:56:45, 19:02:30, and 19:04:26, and the F-09 windows) failed to open the channel at all (`0xE00002BC`), so nothing was sent. My hypothesis, recorded in the F-09 protocol header, was that the 18:15 terminal response occurred because the offer arrived outside the window in which the bud accepts offers; that hypothesis is reported here as a hypothesis, not a finding.

7. Update Initiation and the Workaround Boundary

7.1 Available host tools

The open-source `airpods-fw` utility can inspect known device firmware, retrieve Apple release and catalog metadata, inspect local asset records, request an accessory scan, and observe macOS updater logs [2]. Its `update` implementation verifies a launchd notification registration and executes `notifyutil -p com.apple.SoftwareUpdate.ScanTriggered`. The source does not implement an image-transfer operation or device-specific flashing session. The `--device` flag limits the tool's displayed or monitored target; the Apple notification is system-wide. Its `--watch` mode also requires a fresh connected-device version report to declare success, which can be unavailable for most of the closed-case interval.

The installed `pods-control` utility pertains to audio accessory settings and is not a firmware updater. A payload-inspection utility can parse image metadata but cannot by that fact deliver an authenticated image to hardware. The research distinguishes observation utilities, scan triggers, and actual firmware-transfer mechanisms because confusing these capabilities would create a false positive in the experimental outcome.

The bounded direct-transport series adds the corresponding boundary on the other side: even where a raw channel exists, the information-only sessions stopped before any offer-bearing stage, and the three authorized offer sessions (F-08T/U/V) stopped before any image-data request, transfer, staging, or apply message; in each session the client recorded an offer acknowledgment followed by a terminal processing notification, with the F-08U/V status still ambiguous. The presence of an Apple-signed cached image on the host plus an open control channel is therefore still not an update path; transport, protocol acceptance, and policy all remain separate stages.

7.2 Corrected case-closed scan experiment

The corrected protocol separates an initial evidence snapshot, a system-wide scan notification, a case-closed observation interval, and subsequent live verification. In the reported trial, I ran the dry-run and scan commands directly; the public script codifies the same sequence for replication and records timestamps when used. It posts no more than one notification per invocation. It does not select a firmware file, write a system cache, or bypass a device trust mechanism.

1. Record the paired target `0x201B`, its cached firmware version, and the date. Ensure the Mac has Bluetooth

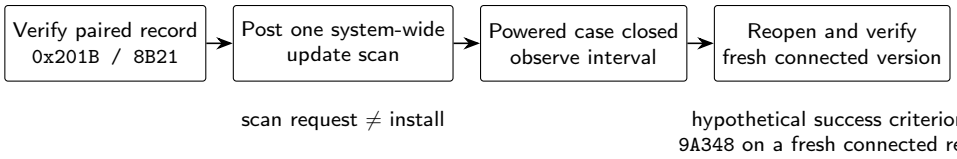


Figure 8: *Corrected update experiment. A cached paired-device record is sufficient to select a local research target; it is not treated as proof that the peripheral is currently reachable. The only requested action is the Apple scan notification.*

and Wi-Fi enabled. If a live reading is accessible before closing the lid, preserve that observation.

2. Place the surviving left earbud in its original charging case, connect the case to USB power, close the lid, and keep the case within Bluetooth range of the Mac.
3. Post one Apple system-wide accessory scan using `python3 airpods-fw update --device 0x201B`. Capture the exit code and exact local timestamp.
4. Leave the case closed for at least 30 minutes, consistent with Apple's public charging-case update sequence [1]. Do not require a continuous active audio connection during this interval.
5. Open the case, reconnect the target, and obtain fresh firmware state from `system_profiler` and `airpods-fw status --device 0x201B`. Preserve whether the readback is connected or cached.
6. Extract a bounded macOS updater log window around the scan timestamp. Remove serial numbers, MAC addresses, pairing tokens, and unrelated devices before public release.

Outcome rule. A fresh connected report of `9A348` for product `0x201B` constitutes host-confirmed version progress. A posted notification followed by a fresh connected `8B21` readback is classified as *no version progression observed in the trial*. A cached-only readback remains indeterminate. A trace containing a targeted eligibility denial or transfer error should be recorded verbatim and evaluated on its own terms. A generalized single-earbud prohibition requires evidence beyond one case.

Current disposition. The manual corrected scan request was posted successfully. At 16:49:43, the connected A3056 and case remained on `8B21`; `system_profiler` corroborated this version and connection state. The intervention result is therefore recorded as no observed version progression, with the updater eligibility and transfer mechanism unresolved.

8. Interpretation and Validity Threats

8.1 Why a local asset is not a firmware installation

The difference between `9A348` in the host cache and `8B21` in accessory records is a useful finding precisely because it isolates a stage boundary. The host can store an image without having selected a particular target, negotiated transport, passed eligibility, written an image, or verified activation. The observed `UARPAssetCacheRecord` and `UARPEndpointPersonalityMobileAsset` messages do not contain a transfer acknowledgement. The inference *firmware exists, therefore the earbud must be able to install it* is not warranted.

8.2 Missing right member as a hypothesis

The lost right earbud is the most salient difference between this configuration and a standard complete pair. That makes a companion-dependency hypothesis worth testing. However, the current data include no Apple-recorded rejection reason tying update refusal to the absent member, no successful paired A3056 control under comparable conditions, and no proof that all normal host conditions were satisfied during previous extended charging. The post-scan connected readback strengthens the observation of a persistent version gap. The missing-earbud mechanism remains a research hypothesis with a clear next experiment, not a verified policy.

The adjacent-hardware evidence sharpens rather than resolves H1. On AirPods Pro 2 (USB-C), the case-mediated completion path demonstrably requires both buds, while the host-driven staging path demonstrably does not [3]. If macOS's updater relied on case coordination for a one-bud A3056 set, a missing companion could plausibly stall completion; if it uses the host path, the missing companion is not architecturally a necessary blocker. Calvados also documents applying a staged image on its primary bud without a synchronized secondary-bud update; this sibling-hardware behavior does not establish an A3056 capability or an operation performed here [3]. Which path macOS attempts for this configuration is exactly the eligibility question this study marks unresolved, and no A3056-specific evidence exists either way.

8.3 Transport gating as a bounded observation

Calvados documents differences between AAP-tunneled session holds and GATT behavior on AirPods Pro 2, but also demonstrates that GATT UARP can successfully carry firmware data on that sibling model [3]. Accordingly, the present A3056 observations cannot establish that the AAP-tunneled channel is inherently privileged for information queries. This study examined the Apple-host path under bounded conditions. The control channel was unreachable from a connected audio state across every variant; the minimal F-12B session that did open produced no UARP answer; and the later, fuller F-08T/U/V sessions on the same channel were answered end to end, including device-confirmed version readbacks, which indicates that the F-12B silence reflects the minimal session the F-12B probe offered rather than fundamental reachability. The F-05E GATT silence (two ATT acknowledgments, zero UARP indications) cannot distinguish missing response framing, an incomplete session, and other transport or firmware-state conditions. Calvados documents a required one-byte fragment prefix on its sibling-hardware GATT UARP channel. The original A3056 GATT probe source was not deposited, so the presence and correctness of that prefix cannot be verified from the public CSV. Three readings of the F-12B result remain possible: the accessory's UARP endpoint requires a session-establishment step that probe did not implement; the endpoint answers only sessions initiated by Apple's own updater; or the endpoint answered and the reply was lost at a layer the probe did not instrument. The F-08T/U/V sessions, which implemented the fuller handshake, were answered; the evidence does not select among the remaining explanations for the F-12B window specifically. What the series establishes is narrower and still useful: a single-bud A3056 exposes an identity-verifiable GATT surface, acknowledges information writes on it, exhibits state-dependent control-channel reachability, and conducts an answered UARP v4 session on the AAP-tunneled channel, without proceeding to image-data requests, transfer, staging, or Apply in the recorded offer sessions.

8.4 Offer acknowledgments, ambiguous terminal flags, and unresolved state

The F-08T/U/V sessions move the evidence boundary: the client recorded accessory-originated UARP running-version and staged-version responses, one acknowledged firmware offer per session, and a subsequent terminal processing notification before any image-data request. In F-08T the client reported abandonment with reason 0. In F-08U and F-08V the recorded flags were `0x0003`; the local decoder labeled them abandoned,

while Calvados also identifies `0x0003` as a possible denied status with reason 1 on sibling hardware [3]. The value `0x0003` alone cannot distinguish these two bitwise encodings. Original decoder code, state context, and independent packet evidence are needed for stronger classification, and even those may not resolve the collision without additional context. Both F-08U/V offers followed a settle gate of at least six seconds. The accessory never requested image data, so the transfer stage was never reached; apply was never sent; the installed image was untouched. These are the only recorded terminal offer-processing notifications in the bounded series. Earlier accessory-originated events, including GATT write acknowledgments, UARP information responses, and offer acknowledgments, had already been observed. The raw terminal values alone do not establish why processing stopped: it does not by itself establish an eligibility policy, a companion-bud dependency, a signature rejection, or an offer-window condition. The recorded reason fields do not discriminate among offer-window timing, companion-bud policy, case or battery gating, a model-specific filter, an implementation or decoding defect, and stale UARP session state. On AirPods Pro 2, Calvados observed that closing and reopening a charging case reset offer state and changed a refused offer into an accepted one [3]. The present three offers occurred close together without a documented equivalent state-reset control, so that alternative remains open; the two settle gates do not test it. The cause and any associated eligibility decision remain unresolved. What the sessions do exclude is narrower: a control-channel transport failure is not the blocker for session establishment and control-message exchange when the channel is open and answered, and the specific asset-integrity failures tested in F-04 are not supported by its structural findings. The image-transfer path itself was not exercised, so no downstream transfer failure mode is excluded. Resolution would require original raw-frame and decoder reconciliation, a separately authorized state-reset comparison under controlled charging and case conditions, or a vendor eligibility statement. No such additional experiment was performed here.

8.5 Sampling, cached data, and tool measurement

The 60 ANC samples were repeated host observations within a small number of test sessions, not 60 independent devices or 60 randomized trials. Their aggregate supports the narrow statement that the host repeatedly reported different settings after different intervention methods. It cannot estimate a population failure rate. The reconnect prompts record operator actions; the return key alone cannot prove Bluetooth attachment, which is why two attempts were excluded. Similarly, a disconnected host record may be stale. The final connected `airpods-fw` and `system_profiler` readings reduce this uncertainty for the bounded trial, but may still share host firmware metadata. A first-party GUI control is not a calibrated external microphone measurement of ANC output.

8.6 Reported ear discomfort and physical exposure

I reported ear discomfort and headaches during use and chose to avoid further wear. Firmware affects software-controlled audio behavior, so a relationship can be considered as a testable question. The current study did not measure acoustic output, eardrum sound-pressure exposure, fit, temperature, radiofrequency transmission power, specific absorption rate, or physiological response. Bluetooth RSSI is received signal strength at the Mac, not a measurement of power absorbed by tissue. The symptom report is recorded as motivation and a user observation; its cause is outside the present experimental result.

8.7 Other limitations

The test uses one surviving left unit, one associated case, and one Mac. I do not have a complete-pair A3056 comparator, device firmware debug output, a known-good service update, a repeated trial across different hosts, or full unredacted internal eligibility messages. Logs were collected after some events

and system privacy redaction obscured the underlying filename and error of numerous pruner messages. Publication timestamps and file modification dates describe host-side artifacts, not peripheral installation times. The manual F-02 scan timestamp was not preserved, so that trial's dwell time is unauditable; the direct-transport series partially compensates by bounding what the transport layer would have shown, but it cannot retroactively complete the scan experiment. Available client-decoded messages do not constitute an independent full HCI packet capture. These constraints determine the scope of the conclusions.

9. Research Implications and Follow-up Work

The main technical gap is observability. Users can see an installed version and Apple publishes a newer version, but normal interfaces do not provide a detailed state history such as *asset downloaded*, *candidate selected*, *target ineligible*, *transfer deferred*, or *activation complete*. This matters whenever a user has a partially functional product, including a single surviving earbud, because an unresolved version difference is difficult to diagnose without service-level tools.

A maintainability improvement would be a documented policy for updating one remaining AirPod, together with an explicit per-device reason if the update cannot proceed. A service diagnostic could report which component is missing, the candidate firmware release, and whether pairing, case state, battery, or transport eligibility prevents the operation. A user-visible last-attempt timestamp would distinguish a failure to initiate from a later transfer failure. Apple's documented service-update appointment path is the correct channel for a surviving single bud today, and whether that path accepts one is an open, vendor-answerable question [1, 6].

Future tests should compare a complete AirPods 4 ANC pair with a deliberately isolated single member under otherwise identical conditions, using only owner-controlled hardware. A replicating investigator should record initial and final connected versions, case power, lid state, host OS version, Bluetooth proximity, time spent closed, and target-correlated UARP events. Repeated conditions should vary one prerequisite at a time. An iPhone-hosted run of Apple's documented procedure would remove the macOS host as a confound. If Apple can supply a device-specific eligibility result or a supported single-earbud service update, that observation would resolve the leading hypothesis far more efficiently than repeated scan notifications.

10. Conclusion

The surviving AirPods 4 ANC A3056 continued to report firmware `8B21` while Apple's current model-family release was `9A348`. macOS retained the newer A3056 firmware asset and registered the earbud and case as updater endpoints. A first scan trial was invalidated by an overly restrictive host-side connection preflight. The corrected manual sequence posted a system-wide scan request; the preserved transcript did not record the scan timestamp, so the case-closed dwell time cannot be reconstructed, and the trial is classified as no observed version progression with unauditable dwell. Two cached reads and a 16:49:43 connected read still showed `8B21`; a second host interface corroborated the connected result. Earlier ANC tests demonstrated host-state differences between intervention methods but did not isolate firmware as their cause.

The bounded direct-transport series then established, without ever transferring firmware, that the accessory is identity-verifiable over GATT, acknowledges UARP information writes on the GATT path, opens its L2CAP AAP control channel only from a paired-but-disconnected state, and, in three bounded authorized sessions, reported running firmware `8.1.21.0` with a staged-version readback of `0.0.0.0` at query time, acknowledged an offer referencing the cached `9A348` build, and produced client-decoded terminal processing notifications before requesting any image data. The raw `0x0003` flags in F-08U/V admit both abandoned and denied interpretations under published sibling-hardware decoding notes. The reviewed evidence contains no

confirmed A3056-correlated transfer, activation, or eligibility verdict; the terminal classification and cause remain unresolved.

The experiment establishes a persistent firmware-version gap following a successfully posted host scan and a connected verification, plus state-dependent accessory transport behavior and accessory-originated offer acknowledgments followed by client-decoded terminal responses. Within the direct-offer path, the observed boundary is sharp: the accessory acknowledged each offer and emitted a terminal processing response at or after offer evaluation and before any recorded transfer request; the two `0x0003` notifications remain ambiguously decoded. Whether Apple's normal updater for this configuration reaches the same stage, or stops earlier for a different reason, is not established by this evidence. Resolving it requires a target-specific Apple eligibility or transfer diagnostic, a vendor service-path statement for a single surviving bud, a matched comparison isolating the second earbud, or an instrumented repetition of the bounded offer session that discriminates among the candidate explanations for these terminal outcomes, including stale session state and protocol decoding.

Data Availability and Reproducibility

The publication supplement contains the LaTeX manuscript source, de-identified ANC observations, selected sanitized UARP events, a consolidated direct-transport run table, a one-scan update protocol, and an explicit provenance inventory. The package is mirrored in a GitHub repository that will be made public upon Zenodo deposit, and this preprint is intended for deposit on Zenodo, where versioned updates should be issued as new versions rather than replaced files [7]. Original macOS diagnostics contain device identifiers and remain private. The public package supports recomputable counts and data summaries, reusable diagnostic procedures, and sanitized experimental outcomes. Protocol-level reproduction of the direct-transport offer sessions additionally requires the native probe and decoder implementation and the private original transcripts, which are not part of this package. The F-02 connected outcome is preserved in a separate public CSV containing six evidence records with time-precision labels. The transport-trial CSV contains 22 consolidated records representing protocol sessions, grouped attempts, and host-state observations, not 22 independent experiments. Future results should be appended as dated experiments rather than replacing this observation.

Independence and Research Ethics

The work was conducted independently using the author's own accessory and Mac, with no third-party device manipulation. The study requested host accessory scans using an existing operating-system interface and did not attempt authenticated firmware bypass, arbitrary device writes, or modifications to Apple-distributed assets. Across every direct-transport run, no image transfer, staging, or apply/reboot command was transmitted; information-only runs transmitted no offer; the three authorized offer sessions each transmitted one offer referencing the cached candidate image, followed by offer acknowledgment and client-decoded terminal processing events (with ambiguous `0x0003` classification in F-08U/V), with no image data requested or sent; and the cached Apple-signed firmware image was never modified or redistributed. The earbud was not required to be worn during firmware experiments. The reported symptoms are not a clinical finding. The title-page cutaway is an AI-generated conceptual illustration, not a device measurement, radiograph, or representation of verified A3056 internal anatomy.

Acknowledgment

The author acknowledges the availability of Apple's public firmware reference, the open-source `airpods-fw` project, and the Calvados and LibrePods projects' published analysis as independent reference material. Inde-

pendent technical review is also acknowledged. All responsibility for the experiment design, interpretation, and publication rests with the author.

References

[1] Apple Inc. "About firmware updates for AirPods." Apple Support, document 106340. Accessed November 7, 2026. https://support.apple.com/en-us/106340.

[2] mooons. "airpods-fw: macOS CLI to inspect AirPods firmware and request Apple's accessory-update scan." Source repository, accessed November 7, 2026. https://github.com/mooons/airpods-fw.

[3] kronerm. "Calvados: AirPods firmware research and tooling, including static analysis of build 9A348 (documentation in docs/firmware.md) and a reported AirPods Pro 2 (USB-C) update from 7.1.21 to 9.0.348." Source repository, pinned to commit caa1113, accessed November 2026. https://github.com/kronerm/calvados/blob/caa1113/docs/uarp.md and https://github.com/kronerm/calvados/blob/caa1113/docs/firmware.md.

[4] LibrePods project. "Issue #612: community discussion of non-Apple-host AirPods firmware updates, including an October 27, 2026 comment reporting an AirPods Pro 2 (USB-C) update and a separate warning about a reported device-bricking incident." Accessed November 2026. https://github.com/librepods-org/librepods/issues/612.

[5] Apple Inc. "Charge your AirPods." Apple Support, document 119912. Accessed November 7, 2026. https://support.apple.com/en-us/119912.

[6] Apple Inc. "Apple Service and Repair for AirPods." Apple Support. Accessed November 7, 2026. https://support.apple.com/airpods/repair.

[7] Zenodo. "FAQ: Versioning." Accessed November 2026. https://zenodo.org/help/versioning.

A. Reproducible Command Inventory

The commands are presented for analyst reproduction, not as evidence of execution in any environment other than the reported case. macOS may restrict fields or redact messages across versions. The specified device product ID is a filter, not an instruction to flash firmware.

A.1 Baseline identification

```
system_profiler SPBluetoothDataType
python3 "$HOME/Downloads/airpods-fw-main/airpods-fw" \
  status --device 0x201B
```

A.2 Public and local firmware state

```
FW="$HOME/Downloads/airpods-fw-main/airpods-fw"
python3 "$FW" check --device 0x201B
python3 "$FW" assets --local --model A3056
```

A.3 One controlled system-wide scan request

```
FW="$HOME/Downloads/airpods-fw-main/airpods-fw"
python3 "$FW" update --device 0x201B --dry-run
python3 "$FW" update --device 0x201B
# Leave the USB-powered case closed for >=30 minutes.
# Then open, reconnect, and check the target.
python3 "$FW" status --device 0x201B
```

A.4 Read-only accessory identity over GATT

```
# Discover, connect, enumerate services, then read only:
# 2A50 PnP ID, 2A29 manufacturer, 2A24 model.
# Never read 2A25 (serial number) for a public artifact.
# The UARP characteristic is identified by service
# 4715650B-5E9D-4AC2-B898-A4FC0AA5DF78 (observed on A3056).
```

A.5 Log scope and privacy

```
python3 "$FW" logs --last 30m --debug
# Raw output can include serial numbers and MAC addresses.
# Keep the raw trace private; create a reviewed public extract.
```

A.6 Interpretation of version results

Table 5: *Outcome classification for one controlled update attempt.*

Result	Classification
Connected target `0x201B` reports `9A348`	Host-confirmed firmware version progress.
Connected target remains `8B21`	No version progress observed during the bounded trial.
Only disconnected, cached `8B21` is available	Installation status indeterminate; obtain live verification.
Scan reports success but no device-state result	Scan requested, outcome unresolved.
Apple emits target-correlated eligibility or transfer error	Preserve exact error, then evaluate its meaning.

B. Selected Sanitized Host Records and Counts

Selected sanitized update record:

```
UARPAssetCacheRecord: 9.0.348.0
.../legacy/A3056/9.0.348.0/prod/firmware.uarp
UARPEndpointPersonalityMobileAsset:
  modelNumber=A3056, fusing=prod,
  fwVersion=8.1.21.0, name=[REDACTED]
```

Count audit: 1,515 expired-file pruner messages + 1,531 other broad keyword matches + 211 unmatched records = 3,257 log records. These are category counts, not independent device update attempts. Across the three ANC observation conditions, 24 + 24 + 12 = 60 host-state samples. The F-02 firmware outcome comes from one scan intervention and one subsequent connected verification session, not independent repeated update attempts. The November 6 historical corpus totals approximately 1,467,868 lines across two overlapping captures (550 + 128 explicit `8B21` mentions; 4,291 + 88 `periodicFirmwareCheck` keyword matches; zero `9A348` mentions). The public transport CSV contains exactly 22 consolidated rows: six F-05-family rows (the F-05 discovery row itself summarizes two scans), one F-04 static-parse row, one F-06 environment-boundary row, three F-08T/U/V offer rows, one F-08X gate row, one grouped later-window row, one grouped F-09 row, one F-10 row, one F-11 row, one F-12 precondition row, two F-12B rows, one grouped F-13 row, one phantom-telemetry row, and one end-state inventory row. This is a row inventory, not a count of independent protocol trials or independent devices. Gate stops and grouped attempts are not counted a second time. The three authorized offer rows each record one offer and no image bytes; terminal flags in F-08U/V remain ambiguously interpreted, and no run sent staging or Apply.

Table 6: *Selected dated events from the source transcript.*

Time (CST)	Subsystem or method	Observation
13:37	`system_profiler`	Connected `0x201B`; left and case `8B21`.
15:09:51	CLI setting request	ANC Off returned immediately after apply.
15:10:06–15:12:06	Passive ANC observation	24/24 returned Noise Cancellation.
15:34:28–15:35:28	Native-setting follow-up	24/24 returned Off.
15:41:22–15:41:48	Reconnect follow-up	12/12 returned Off.
15:49:59	`uarpassetmanagerd`	A3056 `9.0.348.0` candidate cache record found.
15:50:51	`uarpassetmanagerd`	A3056 earbud and case records at `8.1.21.0`; candidate cache available.
16:17:47	Controlled scan script	Cached/disconnected preflight; no notification posted.
Between trials	Manual updater command	Dry run passed; system-wide scan notification posted.
16:47:33	`airpods-fw`	Post-scan check cached/disconnected; earbud and case `8B21`.
16:48:27	`airpods-fw`	Another cached/disconnected check; `8B21`.
16:49:43	`airpods-fw`	Connected target still `8B21`; left and case 100% battery.
After 16:49:43	`system_profiler`	Connected `0x201B`; earbud and case `8B21`.
17:21–17:39	F-05/B/C read-only GATT	Identity, UARP service presence, PnP/model match.
17:35	F-05D run 1	Six UARP information frames TX (tags 4–8 family); zero replies.
17:38	F-05D run 2	Two candidates; uniqueness gate; stopped before writes.
17:39	F-05E UARP writes	2 writes ACKed; 0 indications; no reply.
18:15:09	F-08T authorized offer session	UARP v4 session answered; running `8.1.21.0`, staged `0.0.0.0` confirmed by accessory; offer acknowledged; client decoded abandoned (reason 0); channel closed 7.7 s; no image data requested.
18:29:03	F-08U authorized offer session	AACP settle gate ($\geq$6 s); offer acknowledged; terminal flags `0x0003` (denied/abandoned ambiguous).
18:30:33	F-08V authorized offer session	Same structure and raw terminal flags as F-08U; no image-data request.
$\sim$18:40	`system_profiler`	Phantom *Right Battery Level: 57%* for the lost bud; motivated right-bud-gated F-08X design.
18:56–19:04	F-08T later windows / F-09	L2CAP `0xE00002BC` from connected state; nothing sent.
18:59	F-08X gate	Zero connected records; stopped before transport.
18:59–19:00	F-10 reconnect	Baseband connect `0xE00002D6`; no offer.
19:07	F-11 single open	L2CAP `0xE00002BC`; no retry.
19:12	F-12B run 1	Channel open (MTU 672); handshake send `0xE00002E7`.
19:13	F-12B run 2	Handshake ACK; UARP sync/version TX; channel closed 21.5 s; no reply.
19:17–19:49	F-13 lid-closed waits	Target stayed CONNECTED past 120 s; no traffic.
19:30–19:44	`system_profiler`	End-state: `0x201B` connected `8B21`; `0x2019` separate, `9A348`.

C. Disclosure and Replication Questions for Apple

The following questions are actionable without an allegation of misconduct or a presumed firmware defect.

1. Is AirPods 4 ANC firmware installation supported when the right earbud is missing and the original left A3056 and charging case remain?
2. Does the updater require both buds to be present for eligibility, installation sequencing, validation, or activation? If yes, is this a general rule or version-dependent?
3. Apple documents an Apple Store or Apple Authorized Service Provider firmware-update appointment for users without a nearby Apple device. Does that service path support updating a surviving single earbud, and if so, under what conditions?
4. Which native diagnostic distinguishes a deferred update from an eligibility denial, missing host contact, firmware-channel mismatch, and transfer failure?
5. Can a support technician check why the host finds `9A348` while the target continues to report `8B21`?